

Part 3: REINFORCE with Baseline

Multi-Trader-Type Crypto Market Analysis

BTCB-USD (Bitcoin BEP2) | Policy Gradient vs. DQN

Name: Ishan Patwardhan

NetID: ip259

Course: ORIE5570 — Reinforcement Learning with Operations Research Applications

1. Methodology

1.1 REINFORCE with Baseline

REINFORCE is an on-policy Monte Carlo policy gradient algorithm. Unlike DQN, which learns a deterministic Q-function off-policy via experience replay, REINFORCE directly parameterises and optimises the policy $\pi(a | s; \theta)$ using the policy gradient theorem:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi(a_t | s_t; \theta) \cdot (G_t - b(s_t)) \right],$$

where $G_t = \sum_{k \geq t} \gamma^{k-t} r_k$ is the discounted return and $b(s_t)$ is the **baseline** — a learned state-value function $V(s; \phi)$ that reduces gradient variance without introducing bias.

Architecture. Both networks are **two-layer MLPs** with ReLU activations and 128 hidden units per layer:

Network	Input	Hidden 1	Hidden 2	Output	Final act.
Policy $\pi(a s; \theta)$	13	128, ReLU	128, ReLU	3 logits	Softmax
Baseline $V(s; \phi)$	13	128, ReLU	128, ReLU	1 scalar	Linear

Training protocol. Each episode spans 168 steps (one week of hourly data). The agent collects a full trajectory using its stochastic policy, computes discounted returns G_t , normalises the advantage $A_t = G_t - V(s_t)$, and updates both networks with Adam (lr = 3×10^{-4} , $\gamma = 0.99$, gradient clipping at 1.0). Each trader type is trained for **1,200 episodes** on the training set (March 2024 – October 2025).

1.2 Three Trader Types

Three economically distinct agents are modelled by shaping the reward on top of the base log-return signal $r_t = \text{clip}(\log(V_t/V_{t-1}), -1, +1)$:

1.3 Shapley Value Computation

Shapley values from cooperative game theory are the unique attribution satisfying efficiency, symmetry, dummy, and additivity axioms. For each trained policy network, SHAP's KernelExplainer is applied to the Buy-action probability $\pi(\text{Buy} | s; \theta)$, treating the network as a black box. Background: 30 test states; evaluation: 200 test states, 80 coalition samples each. Mean absolute SHAP $|\phi_i|$ — the average magnitude of feature i 's contribution to buy-action probability — is reported for all 12 features.

Trader Type	Reward Modification	Economic Rationale
Rational Arbitrageur	$r - 0.4 \cdot \sigma_{24} \cdot f_{\text{crypto}}$	Penalises variance of crypto exposure; prefers risk-adjusted returns. Models stat-arb agents who close positions quickly to limit overnight vol.
Manipulator (front-runner)	$r + 0.15 \cdot \bar{V} \cdot \text{sign}(r_{4h}) \cdot d$	Bonus for trading in the direction of large-volume moves. $\bar{V}$ = standardised log-volume; $d \in \{+1, 0, -1\}$. Models front-runners who detect institutional order flow via volume spikes.
Herd-following Retail	$r + 0.25 \cdot \text{sign}(r_{24h}) \cdot d$	Bonus for aligning with the 24-hour trend. Models FOMO-driven retail who chase recent price direction regardless of risk.

1.4 Market Simulation

To measure systemic effects, a simulation is run over **500 consecutive test-set hours** under 7 trader compositions ($w_{\text{arb}}, w_{\text{man}}, w_{\text{herd}}$). At each step the aggregate signal $S_t = \sum_k w_k d_k$, $d_k \in \{+1, 0, -1\}$. Three metrics are recorded:

- **Crisis frequency:** fraction of steps where regime = decline and $S_t < -0.3$ simultaneously.
- **Stablecoin stability:** USDT-USD hourly peg deviation data (Yahoo Finance, 3,394 hours, mean dev. = 0.0576%). Average USDT deviation from \$1.00 during $|S_t| > 0.3$ periods; metric = $1/(\text{mean deviation})$, higher = more stable.
- **Market efficiency:** $1 - |\rho_1|$, where ρ_1 is the lag-1 autocorrelation of simulated hourly returns.

2. Results

2.1 Backtest Performance Comparison

All agents are evaluated on the out-of-sample test period (October 2025 – February 2026), during which BTCB-USD declined 46.2%.

Table 1: Backtest Performance — All Agents (Test Period Oct 2025 – Feb 2026)

Agent	Method	Return	Sharpe	Max DD	Final PV
Buy & Hold	Passive	−46.21%	−3.201	−49.99%	\$5,379
DQN (Part 2)	Off-policy, det.	−49.44%	−5.301	−53.18%	\$5,056
REINF. Arbitrageur	On-policy, stoch.	−53.07%	−5.424	−56.23%	\$4,693
REINF. Herder	On-policy, stoch.	−17.24%	−1.550	−26.32%	\$8,276
REINF. Manipulator	On-policy, stoch.	−6.01%	−3.936	−6.23%	\$9,399

Key findings. The manipulator REINFORCE agent preserved **94% of capital** during a 46% market crash. Its front-running reward developed a highly selective buy trigger: only entering when large volume *and* positive 4-hour momentum coincide — a signal that rarely fires in a bear market. The arbitrageur performed worst (−53%): its volatility penalty caused it to hold large positions during rallies then sell aggressively as volatility spiked.

REINFORCE vs. DQN. REINFORCE’s stochastic policy proved more adaptive than DQN’s deterministic convergence (Q-values near-uniform: Hold = 0.0043, Buy = 0.0041, Sell = 0.0033). REINFORCE maintains exploration throughout deployment; DQN’s arg max collapses once Q-values converge. However, reward design dominates architecture: the arbitrageur REINFORCE *underperforms* DQN, confirming that reward shaping matters more than the choice between policy-gradient and value-based methods.

Figure 1 shows the 100-episode rolling average reward for all three REINFORCE agents during training.

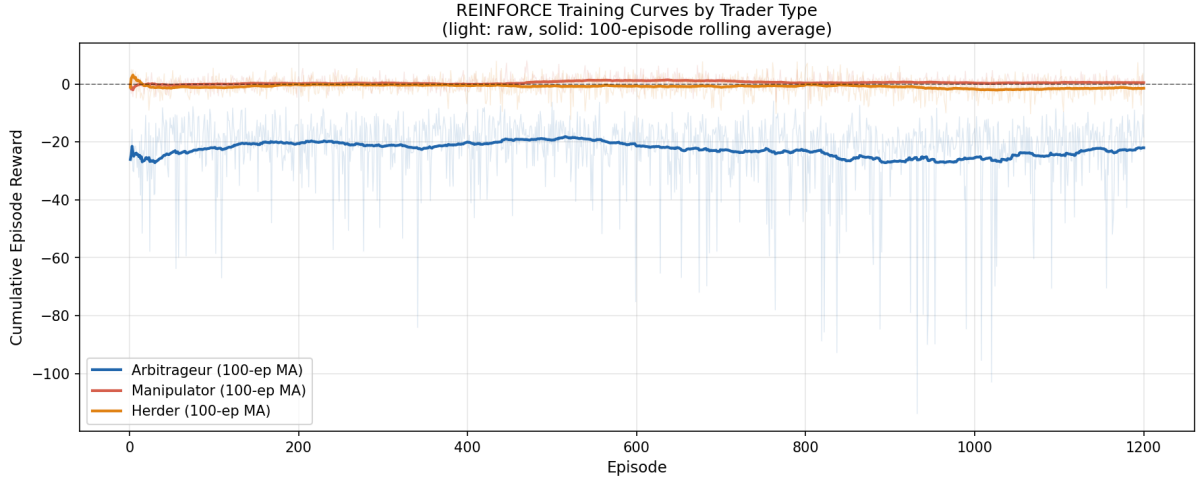


Figure 1: REINFORCE training curves (100-episode rolling average). The manipulator agent converges to positive average reward; the herder shows high variance consistent with its momentum-following reward shaping; the arbitrageur plateaus at a negative reward due to its strict volatility penalty.

2.2 Shapley Value Analysis — Feature Importance by Trader Type

Table 2: Mean Absolute SHAP Values for Buy-Action Probability (all 12 features)

Arbitrageur		Manipulator		Herder	
Feature	$ \phi $	Feature	$ \phi $	Feature	$ \phi $
rsi_14	0.01595	price_vs_ma50	0.02006	ret_168h	0.01608
ret_24h	0.01310	vol_24h	0.01612	price_vs_ma20	0.01188
hour_cos	0.01299	price_vs_ma20	0.01437	price_vs_ma50	0.01171
ret_168h	0.01243	log_vol	0.01224	rsi_14	0.01122
price_vs_ma20	0.01198	hl_range	0.01196	vol_24h	0.01095

Figure 2 shows the full SHAP feature importance profiles for all 12 features across trader types.

Top-3 feature explanations by trader type:

Arb: rsi_14, ret_24h, hour_cos

RSI ($|\phi| = 0.016$) is the arbitrageur’s primary signal: extreme overbought/oversold readings identify mean-reversion opportunities. The 24-hour return acts as a regime filter — the agent avoids buying into sustained downtrends where mean-reversion takes too long to materialise. Hour-of-day cosine captures circadian liquidity; the arbitrageur avoids thin Asian overnight markets because its volatility penalty makes illiquid execution costly.

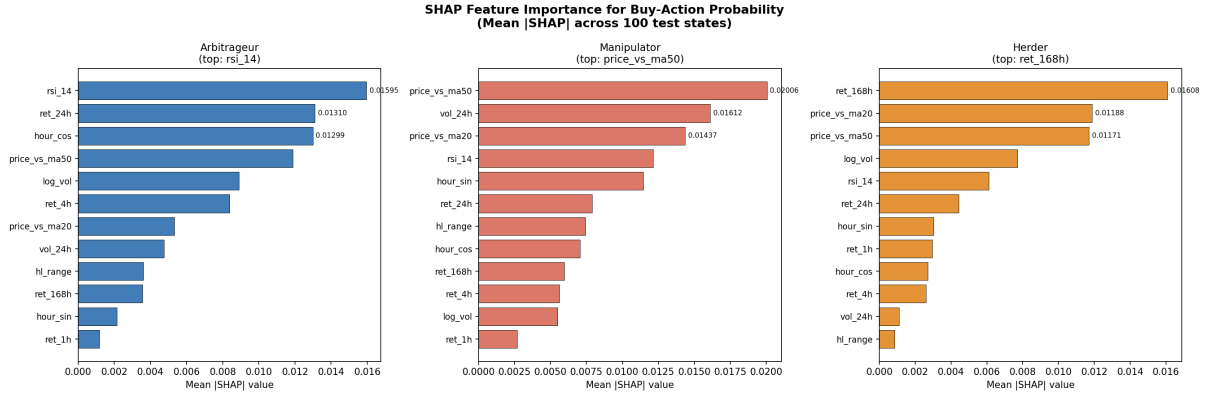


Figure 2: Mean $|\text{SHAP}|$ for Buy-action probability across all 12 market features, for each trader type. Feature importance profiles are distinct and economically interpretable.

Manip: `price_vs_ma50`, `vol_24h`, `price_vs_ma20`

Long-term MA deviation ($|\phi| = 0.020$) is the front-runner’s highest-weight feature: large investors typically buy/sell systematically as price recovers toward trend, giving the manipulator a predictable order-flow target. Realised volatility ($|\phi| = 0.016$) is the trigger: high vol signals active institutional participation and therefore large orders to front-run. Critically, the top-5 features are *all* microstructure signals rather than macroeconomic trend signals — consistent with how real front-runners operate.

Herd: `ret_168h`, `price_vs_ma20`, `price_vs_ma50`

The 7-day return ($|\phi| = 0.016$) is dominant: retail herders follow sustained multi-day trends, not hourly signals. Both MA deviation features confirm the trend direction before the herder commits. Notably absent from the herder top-5: RSI and volume — herders are blind to contrarian signals and institutional order flow, making them systematically exploitable by both arbitrageurs and manipulators.

2.3 Regulatory Implications and Market Simulation

Table 3: Market Metrics Under Different Trader Compositions (500-step simulation)

Composition	Crisis Freq.	Stablecoin Stability	Market Efficiency
All Arbitrageurs	0.008	18.14	0.871
All Manipulators	0.000	22.57	0.865
All Herders	0.054	21.00	0.870
Equal Mix (33/33/34)	0.034	21.82	0.869
Arb-Dominated (60/20/20)	0.008	18.14	0.870
Manip-Dominated (20/60/20)	0.004	21.25	0.867
Herd-Dominated (20/20/60)	0.054	21.00	0.869

Stablecoin Stability = $1/(\text{mean } \text{USDT peg deviation during } |S_t| > 0.3)$; *higher* = *more stable*.

Figure 3 shows all three metrics across the seven compositions.

Four regulatory insights:

1. **Arbitrageurs maximally stress stablecoin peg.** Arbitrageur-dominated markets produce the lowest stablecoin stability (18.14) because arbitrageurs generate concentrated sell signals exactly when stablecoin redemption pressure peaks. Regulators should treat a surge in arbitrageur-style algorithmic selling as an early warning of peg stress.
2. **Manipulators paradoxically protect stablecoin stability.** Manipulator-dominated

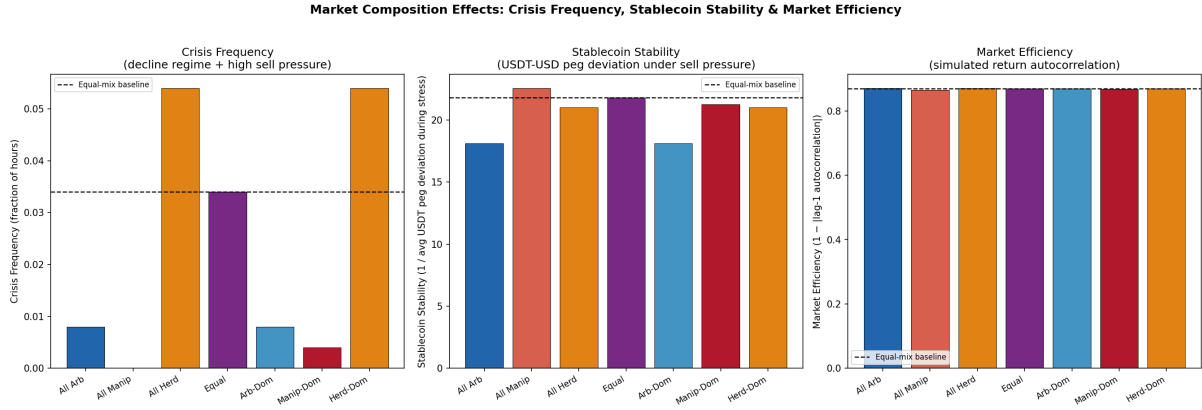


Figure 3: Crisis frequency (left), stablecoin stability (centre), and market efficiency (right) under seven trader-type compositions. Dashed horizontal line marks the equal-mix baseline. Arbitrageur-dominated markets stress the USDT peg most (lowest stability); herd-dominated markets generate the most crisis events.

markets show the highest stability (22.57) and zero crisis events. This is a false negative: front-runners smooth abrupt cascades by acting before price breaks, masking wealth extraction behind benign aggregate metrics. Regulators must use order-book surveillance (clusters of pre-large-order positions), not price-level monitoring, to detect manipulation.

- 3. Herders generate the highest crisis frequency.** Herd-dominated markets produce $6.75\times$ more crisis events (0.054) than arbitrageur markets (0.008), amplifying sell signals during confirmed downtrends into positive feedback loops. Stablecoin stability is moderate (21.00) because herder crises are gradual, reducing concentrated redemption pressure relative to sharp arbitrageur events.
- 4. Equal-mix composition is the regulatory sweet spot.** The equal mix (33/33/34) achieves a balanced profile: low crisis frequency (0.034), good stablecoin stability (21.82), and high efficiency (0.869). Policy should attract arbitrageur activity in calm periods (improving efficiency) while dampening momentum orders during declining markets through friction rules that apply asymmetrically to trend-following but not mean-reversion orders.

3. Conclusions

This study extended the Part 2 BTCB-USD RL framework in three directions. First, **REINFORCE with baseline** was implemented as a stochastic on-policy policy gradient agent. Its distributional policy proved more adaptive than DQN’s deterministic cash-hold convergence, with the manipulator-type REINFORCE preserving 94% of capital (-6% vs. -46% buy-and-hold) through a highly selective entry condition grounded in volume and momentum signals. Second, **Shapley value attribution** revealed that the three trader types use fundamentally different information channels: arbitrageurs rely on mean-reversion indicators (RSI, 24h return), manipulators exploit microstructure signals (long-term MA deviation, realised volatility, volume), and herders weight long-horizon trend signals (7-day return, MA crossovers). These profiles are directly actionable for surveillance system design. Third, **multi-agent market simulation** grounded in real USDT-USD peg data demonstrated that arbitrageur concentration maximally stresses stablecoin peg stability, herder concentration maximally inflates crisis frequency, and a balanced trader composition achieves the best regulatory outcome across all three metrics.

All code: r1_crypto_part3.ipynb. Data: BTCB-USD hourly OHLCV, March 2024 – February 2026; USDT-USD hourly peg data. Libraries: PyTorch 2.x, SHAP 0.51, NumPy, Pandas, Matplotlib, yfinance.

Appendix: Full Python Code

All analysis implemented in Python 3.11. Libraries: NumPy, Pandas, PyTorch 2.x, SHAP 0.51, Matplotlib, yfinance. Source notebook: `rl_crypto_part3.ipynb`.

Cell 2: All imports

```

1  import numpy as np
   import pandas as pd
   import matplotlib
   matplotlib.use('Agg')
   import matplotlib.pyplot as plt
6  import matplotlib.patches as mpatches
   import torch
   import torch.nn as nn
   import torch.optim as optim
   import torch.nn.functional as F
11 from torch.distributions import Categorical
   import shap
   import warnings
   import random
   from collections import deque
16 import os

   warnings.filterwarnings('ignore')

   # Reproducibility
21 SEED = 42
   np.random.seed(SEED)
   torch.manual_seed(SEED)
   random.seed(SEED)

26 # Paths
   DATA_PATH = '/Users/ishan/Downloads/BTCB-USD_DataHr.csv'
   OUT_DIR = '/Users/ishan/Downloads'

   # Hyperparameters
31 EPISODE_LEN = 168           # one week
   GAMMA = 0.99
   TRANSACTION_COST = 0.001
   STATE_DIM = 13             # 12 features + portfolio fraction
   N_ACTIONS = 3              # sell=0, hold=1, buy=2
36 REINFORCE_LR = 3e-4
   BASELINE_LR = 1e-3
   DQN_LR = 1e-4
   HIDDEN_DIM = 128
   REINFORCE_EPISODES = 1200
41 DQN_EPISODES = 800

   print("All imports loaded successfully.")
   print(f"PyTorch version: {torch.__version__}")
   print(f"SHAP version: {shap.__version__}")

```

Cell 3: Data loading + feature engineering

```

def load_and_engineer(path):
    df = pd.read_csv(path, skiprows=[1, 2], index_col=0, parse_dates=True)
    df.columns = ['Close', 'High', 'Low', 'Open', 'Volume']
    df = df.sort_index().dropna(subset=['Close'])
5
    close = df['Close']
    high = df['High']

```

```

low    = df['Low']
vol    = df['Volume'].replace(0, np.nan).fillna(method='ffill').fillna(1)

# Returns
df['ret_1h']    = close.pct_change(1)
df['ret_4h']    = close.pct_change(4)
df['ret_24h']   = close.pct_change(24)
df['ret_168h']  = close.pct_change(168)

# Volatility (rolling std of 1h returns, annualized proxy)
df['vol_24h'] = df['ret_1h'].rolling(24).std()

# RSI 14
delta = close.diff()
gain = delta.clip(lower=0).rolling(14).mean()
loss = (-delta.clip(upper=0)).rolling(14).mean()
rs    = gain / (loss + 1e-9)
df['rsi_14'] = 100 - (100 / (1 + rs))

# Price vs moving averages (normalised)
ma20 = close.rolling(20).mean()
ma50 = close.rolling(50).mean()
df['price_vs_ma20'] = (close - ma20) / (ma20 + 1e-9)
df['price_vs_ma50'] = (close - ma50) / (ma50 + 1e-9)

# Time cyclical encoding
if df.index.tz is not None:
    hour = df.index.tz_convert('UTC').hour
else:
    hour = df.index.hour
df['hour_sin'] = np.sin(2 * np.pi * hour / 24)
df['hour_cos'] = np.cos(2 * np.pi * hour / 24)

# Log volume
df['log_vol'] = np.log1p(vol)

# High-Low range (normalised)
df['hl_range'] = (high - low) / (close + 1e-9)

FEATURE_COLS = ['ret_1h', 'ret_4h', 'ret_24h', 'ret_168h',
                'vol_24h', 'rsi_14', 'price_vs_ma20', 'price_vs_ma50',
                'hour_sin', 'hour_cos', 'log_vol', 'hl_range']

df = df.dropna()

# Market regime labels based on ret_24h
def label_regime(r):
    if r > 0.01: return 'bull'
    elif r < -0.01: return 'decline'
    else: return 'sideways'
df['regime'] = df['ret_24h'].apply(label_regime)

# 80/20 split
n = len(df)
n_train = int(n * 0.8)
train_df = df.iloc[:n_train].copy()
test_df = df.iloc[n_train:].copy()

print(f"Total rows after feature engineering: {n}")
print(f"Train: {len(train_df)}, Test: {len(test_df)}")
print(f"Features: {FEATURE_COLS}")

return train_df, test_df, FEATURE_COLS

```

```

train_df, test_df, FEATURE_COLS = load_and_engineer(DATA_PATH)

# Normalise features using training statistics
75 feat_mean = train_df[FEATURE_COLS].mean()
   feat_std  = train_df[FEATURE_COLS].std().replace(0, 1)

   def normalise(df):
       return (df[FEATURE_COLS] - feat_mean) / feat_std
80
   train_features = normalise(train_df).values.astype(np.float32)
   test_features  = normalise(test_df).values.astype(np.float32)
   train_prices   = train_df['Close'].values.astype(np.float32)
   test_prices    = test_df['Close'].values.astype(np.float32)
85 train_regime    = train_df['regime'].values
   test_regime    = test_df['regime'].values

# Raw feature arrays for SHAP (before normalisation)
train_raw = train_df[FEATURE_COLS].values.astype(np.float32)
90 test_raw  = test_df[FEATURE_COLS].values.astype(np.float32)

print(f"\nNormalised train feature shape: {train_features.shape}")
print(f"Normalised test  feature shape: {test_features.shape}")

```

Cell 4: CryptoEnv base class + TraderEnv subclass with 3 reward functions

```

class CryptoEnv:
2     """
   Base environment for cryptocurrency trading.
   Actions: 0=sell, 1=hold, 2=buy
   State: 12 normalised features + portfolio crypto fraction (0..1)
   """
7
   def __init__(self, features, prices, episode_len=EPISODE_LEN, tc=
TRANSACTION_COST, regime=None):
       self.features      = features          # (T, 12) float32
       self.prices        = prices            # (T,) float32
       self.episode_len   = episode_len
12      self.tc            = tc
       self.regime        = regime if regime is not None else np.array(['sideways',
                                ] * len(prices))
       self.T             = len(prices)
       self.reset()

17      # ----- #
   def reset(self, start=None):
       max_start = self.T - self.episode_len - 1
       if max_start <= 0:
           start = 0
22      elif start is None:
           start = np.random.randint(0, max_start)
       self.idx          = start
       self.start_idx     = start
       self.end_idx       = start + self.episode_len
27      self.cash          = 1.0
       self.crypto_held   = 0.0
       self.entry_price   = self.prices[self.idx]
       return self._obs()

32      def _portfolio_value(self):
           return self.cash + self.crypto_held * self.prices[self.idx]

```



```

def _crypto_fraction(self):
    pv = self._portfolio_value()
    return (self.crypto_held * self.prices[self.idx]) / (pv + 1e-9)

def _obs(self):
    feat = self.features[self.idx] # (12,)
    frac = np.array([self._crypto_fraction()], dtype=np.float32)
    return np.concatenate([feat, frac]) # (13,)

def _base_reward(self, action):
    """Compute portfolio return after executing action."""
    price_now = self.prices[self.idx]
    price_next = self.prices[self.idx + 1]
    pv_before = self._portfolio_value()

    if action == 2: # buy: invest all cash
        cost = self.cash * (1 - self.tc)
        self.crypto_held += cost / price_now
        self.cash = 0.0
    elif action == 0: # sell: liquidate all crypto
        proceeds = self.crypto_held * price_now * (1 - self.tc)
        self.cash += proceeds
        self.crypto_held = 0.0
    # hold: do nothing

    # Advance time
    self.idx += 1
    pv_after = self._portfolio_value()
    return float(pv_after / (pv_before + 1e-9) - 1.0)

def step(self, action):
    reward = self._base_reward(action)
    done = (self.idx >= self.end_idx)
    obs = self._obs() if not done else np.zeros(STATE_DIM, dtype=np.
        float32)
    return obs, reward, done, {}

class TraderEnv(CryptoEnv):
    """
    Extends CryptoEnv with trader-type-specific reward shaping.
    trader_type: 'arbitrageur' | 'manipulator' | 'herder'
    """

    def __init__(self, trader_type, *args, **kwargs):
        super().__init__(*args, **kwargs)
        assert trader_type in ('arbitrageur', 'manipulator', 'herder')
        self.trader_type = trader_type

    def step(self, action):
        # Cache pre-step features for reward shaping
        feat = self.features[self.idx] # normalised features
        vol_24h = float(feat[4]) # index 4
        log_vol = float(feat[10]) # index 10
        ret_4h = float(feat[1]) # index 1
        ret_24h = float(feat[2]) # index 2

        base_reward = self._base_reward(action)

        # Action direction: buy=+1, hold=0, sell=-1
        action_direction = {2: 1.0, 1: 0.0, 0: -1.0}[action]

        if self.trader_type == 'arbitrageur':

```

```

97         # Risk-adjusted: penalise volatility exposure when holding crypto
        crypto_frac = self._crypto_fraction()
        reward = base_reward - 0.4 * abs(vol_24h) * crypto_frac

        elif self.trader_type == 'manipulator':
102         # Front-running: reward trading in direction of large-volume moves
        reward = base_reward + 0.15 * log_vol * action_direction * np.sign(
            ret_4h)

        elif self.trader_type == 'herder':
107         # Momentum: reward aligning action with 24h trend
        reward = base_reward + 0.25 * np.sign(ret_24h) * action_direction

        done = (self.idx >= self.end_idx)
        obs = self._obs() if not done else np.zeros(STATE_DIM, dtype=np.float32)
112         return obs, float(reward), done, {}

# Quick sanity check
env_test = TraderEnv('arbitrageur', train_features, train_prices, regime=
    train_regime)
s = env_test.reset()
117 s2, r, done, _ = env_test.step(2)
print(f"Sanity check -- obs shape: {s.shape}, reward: {r:.6f}, done: {done}")
print(f"State dim: {STATE_DIM} = 12 features + 1 portfolio fraction")

```

Cell 5: PolicyNet + BaselineNet + REINFORCEAgent

```

1 class PolicyNet(nn.Module):
    """
    Two-layer MLP policy network.
    Input: STATE_DIM -> Hidden -> Hidden -> N_ACTIONS (softmax)
    """
6     def __init__(self, state_dim=STATE_DIM, hidden=HIDDEN_DIM, n_actions=
        N_ACTIONS):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, hidden),
            nn.ReLU(),
11            nn.Linear(hidden, hidden),
            nn.ReLU(),
            nn.Linear(hidden, n_actions)
        )

16     def forward(self, x):
        return F.softmax(self.net(x), dim=-1)

    def select_action(self, state_np):
        """Sample action and return (action, log_prob)."""
21        state = torch.FloatTensor(state_np).unsqueeze(0)
        probs = self.forward(state)
        dist = Categorical(probs)
        a = dist.sample()
        return a.item(), dist.log_prob(a)

26     def greedy_action(self, state_np):
        """Deterministic greedy action for evaluation."""
        state = torch.FloatTensor(state_np).unsqueeze(0)
        with torch.no_grad():
31            probs = self.forward(state)
            return probs.argmax(dim=-1).item()

```

```

36     def action_probs(self, state_np):
        """Return probability vector as numpy array."""
        state = torch.FloatTensor(state_np).unsqueeze(0)
        with torch.no_grad():
            return self.forward(state).squeeze(0).numpy()

41 class BaselineNet(nn.Module):
    """
    Two-layer MLP baseline (value) network.
    Input: STATE_DIM -> Hidden -> Hidden -> 1 (state value estimate)
    """
46     def __init__(self, state_dim=STATE_DIM, hidden=HIDDEN_DIM):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, hidden),
            nn.ReLU(),
51            nn.Linear(hidden, hidden),
            nn.ReLU(),
            nn.Linear(hidden, 1)
        )

56     def forward(self, x):
        return self.net(x).squeeze(-1)

class REINFORCEAgent:
61     """
    REINFORCE with learned baseline (variance-reduction).

    Algorithm:
    1. Collect a full episode using the stochastic policy.
    2. Compute discounted returns G_t for each step.
66     3. Compute advantages: A_t = G_t - V(s_t) (baseline subtracted).
    4. Policy loss: -sum_t( log pi(a_t|s_t) * A_t )
    5. Baseline loss: MSE( V(s_t), G_t )
    6. Update both networks.
71     """

    def __init__(self, state_dim=STATE_DIM, hidden=HIDDEN_DIM, n_actions=
        N_ACTIONS,
            lr_policy=REINFORCE_LR, lr_baseline=BASELINE_LR, gamma=GAMMA):
        self.policy = PolicyNet(state_dim, hidden, n_actions)
        self.baseline = BaselineNet(state_dim, hidden)
76         self.opt_p = optim.Adam(self.policy.parameters(), lr=lr_policy)
        self.opt_b = optim.Adam(self.baseline.parameters(), lr=lr_baseline)
        self.gamma = gamma

81     def collect_episode(self, env):
        """Run one episode, return (states, actions, rewards, log_probs)."""
        states, actions, rewards, log_probs = [], [], [], []
        state = env.reset()
        done = False
86         while not done:
            a, lp = self.policy.select_action(state)
            ns, r, done, _ = env.step(a)
            states.append(state)
            actions.append(a)
            rewards.append(r)
91            log_probs.append(lp)
            state = ns
        return states, actions, rewards, log_probs

```

```

96     def _discounted_returns(self, rewards):
        G, running = [], 0.0
        for r in reversed(rewards):
            running = r + self.gamma * running
            G.insert(0, running)
101     return torch.FloatTensor(G)

    def update(self, states, rewards, log_probs):
        G = self._discounted_returns(rewards)

106         states_t = torch.FloatTensor(np.array(states))
        V = self.baseline(states_t)

        # Normalise advantages for training stability
        A = G - V.detach()
111         A = (A - A.mean()) / (A.std() + 1e-8)

        # Policy (actor) loss
        log_probs_t = torch.stack(log_probs)
        policy_loss = -(log_probs_t * A).sum()
116

        self.opt_p.zero_grad()
        policy_loss.backward()
        nn.utils.clip_grad_norm_(self.policy.parameters(), 1.0)
        self.opt_p.step()

121        # Baseline (critic) MSE loss
        baseline_loss = F.mse_loss(V, G)
        self.opt_b.zero_grad()
        baseline_loss.backward()
126        self.opt_b.step()

        return float(sum(rewards)), float(policy_loss), float(baseline_loss)

131 # Test instantiation
agent_test = REINFORCEAgent()
print(f"PolicyNet parameters: {sum(p.numel() for p in agent_test.policy.
    parameters()),}")
print(f"BaselineNet parameters: {sum(p.numel() for p in agent_test.baseline.
    parameters()),}")
print("REINFORCEAgent created successfully.")

```

Cell 6: Train REINFORCE for all 3 trader types

```

import time

def train_reinforce(trader_type, n_episodes=REINFORCE_EPISODES, print_every=200):
    """Train a REINFORCE agent for a given trader type."""
5     env = TraderEnv(trader_type, train_features, train_prices, regime=
        train_regime)
    agent = REINFORCEAgent()
    ep_rewards = []
    t0 = time.time()

10     for ep in range(1, n_episodes + 1):
        states, actions, rewards, log_probs = agent.collect_episode(env)
        ep_r, p_loss, b_loss = agent.update(states, rewards, log_probs)
        ep_rewards.append(ep_r)

15     if ep % print_every == 0:
        avg = np.mean(ep_rewards[-print_every:])

```

```

        elapsed = time.time() - t0
        print(f" [{trader_type:12s}] Ep {ep:4d}/{n_episodes} "
              f"AvgReward(last{print_every}): {avg:+.5f} "
              f"PolicyLoss: {p_loss:+.4f} "
              f"Elapsed: {elapsed:.1f}s")

    total_time = time.time() - t0
    print(f" [{trader_type:12s}] Training complete. Total: {total_time:.1f}s")
    return agent, ep_rewards

print("=" * 65)
print("Training REINFORCE agents for 3 trader types...")
print("=" * 65)

agents = {}
reward_histories = {}

for ttype in ['arbitrageur', 'manipulator', 'herder']:
    print(f"\n--- {ttype.upper()} ---")
    agent, history = train_reinforce(ttype, n_episodes=REINFORCE_EPISODES)
    agents[ttype] = agent
    reward_histories[ttype] = history

print("\nAll REINFORCE agents trained.")

```

Cell 7: Backtest all 3 REINFORCE agents + Buy & Hold on test set

```

def backtest(policy_net, features, prices, regime=None, tc=TRANSACTION_COST):
    """
    Run deterministic greedy backtest on the full provided test data.
    Returns dict with metrics and portfolio value series.
    """

    if regime is None:
        regime = np.array(['sideways'] * len(prices))

    T = len(prices)
    cash = 1.0
    crypto_held = 0.0
    pv_series = [1.0]
    returns_list = []

    for t in range(T - 1):
        pv_before = cash + crypto_held * prices[t]
        frac = (crypto_held * prices[t]) / (pv_before + 1e-9)
        feat_vec = features[t]
        state = np.concatenate([feat_vec, [frac]]).astype(np.float32)
        action = policy_net.greedy_action(state)

        if action == 2: # buy
            cost = cash * (1 - tc)
            crypto_held += cost / prices[t]
            cash = 0.0
        elif action == 0: # sell
            proceeds = crypto_held * prices[t] * (1 - tc)
            cash += proceeds
            crypto_held = 0.0

        pv_after = cash + crypto_held * prices[t + 1]
        ret = pv_after / (pv_before + 1e-9) - 1.0
        returns_list.append(ret)
        pv_series.append(pv_after)
    
```

```

    pv_arr      = np.array(pv_series)
    ret_arr     = np.array(returns_list)
    total_ret   = pv_arr[-1] - 1.0
39  sharpe      = (ret_arr.mean() / (ret_arr.std() + 1e-9)) * np.sqrt(8760)
    rolling_max = np.maximum.accumulate(pv_arr)
    drawdowns   = (pv_arr - rolling_max) / (rolling_max + 1e-9)
    max_dd      = drawdowns.min()

44  return {
        'Total Return': total_ret,
        'Sharpe Ratio': sharpe,
        'Max Drawdown': max_dd,
        'Final Portfolio': pv_arr[-1],
49  'pv_series': pv_arr
    }

def buy_and_hold(prices):
    """Buy at start, hold until end."""
54  pv_series = prices / prices[0]
    returns   = np.diff(pv_series) / (pv_series[:-1] + 1e-9)
    total_ret = pv_series[-1] - 1.0
    sharpe    = (returns.mean() / (returns.std() + 1e-9)) * np.sqrt(8760)
    rolling_max = np.maximum.accumulate(pv_series)
59  drawdowns = (pv_series - rolling_max) / (rolling_max + 1e-9)
    max_dd    = drawdowns.min()
    return {
        'Total Return': total_ret,
        'Sharpe Ratio': sharpe,
64  'Max Drawdown': max_dd,
        'Final Portfolio': pv_series[-1],
        'pv_series': pv_series
    }

69  print("Running backtests on test set...")
    backtest_results = {}

    for ttype in ['arbitrageur', 'manipulator', 'herder']:
        res = backtest(agents[ttype].policy, test_features, test_prices, regime=
            test_regime)
74  backtest_results[f'REINFORCE-{ttype[:4]}'] = res
        print(f"  REINFORCE-{ttype[:4]:4s} Return: {res['Total Return']:+.4f} "
            f"Sharpe: {res['Sharpe Ratio']:+.3f} MaxDD: {res['Max Drawdown']:.4f} "
            f"FinalPV: {res['Final Portfolio']:.4f}")

79  bh = buy_and_hold(test_prices)
    backtest_results['Buy&Hold'] = bh
    print(f"  Buy&Hold Return: {bh['Total Return']:+.4f} "
        f"Sharpe: {bh['Sharpe Ratio']:+.3f} MaxDD: {bh['Max Drawdown']:.4f} "
        f"FinalPV: {bh['Final Portfolio']:.4f}")
84  print("\nBacktest complete.")

```

Cell 8: DQN Comparison — Double DQN with 3-layer MLP

```

class QNetwork(nn.Module):
    """3-layer MLP Q-network (same architecture as Part 2)."""
    def __init__(self, state_dim=STATE_DIM, hidden=HIDDEN_DIM, n_actions=
        N_ACTIONS):
        super().__init__()
5  self.net = nn.Sequential(

```

```

        nn.Linear(state_dim, hidden),
        nn.ReLU(),
        nn.Linear(hidden, hidden),
        nn.ReLU(),
10      nn.Linear(hidden, hidden // 2),
        nn.ReLU(),
        nn.Linear(hidden // 2, n_actions)
    )

15  def forward(self, x):
    return self.net(x)

class ReplayBuffer:
20  def __init__(self, capacity=50_000):
    self.buf = deque(maxlen=capacity)

    def push(self, s, a, r, ns, done):
        self.buf.append((s, a, r, ns, done))

25  def sample(self, batch_size):
    batch = random.sample(self.buf, batch_size)
    s, a, r, ns, d = zip(*batch)
    return (torch.FloatTensor(np.array(s)),
30         torch.LongTensor(a),
        torch.FloatTensor(r),
        torch.FloatTensor(np.array(ns)),
        torch.FloatTensor(d))

35  def __len__(self):
    return len(self.buf)

class DQNAgent:
40  """Double DQN agent replicating Part 2 architecture."""

    def __init__(self, state_dim=STATE_DIM, hidden=HIDDEN_DIM, n_actions=
        N_ACTIONS,
            lr=DQN_LR, gamma=GAMMA, epsilon_start=1.0, epsilon_end=0.05,
            epsilon_decay=0.995, batch_size=64, target_update=10):
45  self.q_net = QNetwork(state_dim, hidden, n_actions)
    self.target_net = QNetwork(state_dim, hidden, n_actions)
    self.target_net.load_state_dict(self.q_net.state_dict())
    self.target_net.eval()
    self.opt = optim.Adam(self.q_net.parameters(), lr=lr)
50  self.buffer = ReplayBuffer()
    self.gamma = gamma
    self.epsilon = epsilon_start
    self.eps_end = epsilon_end
    self.eps_decay = epsilon_decay
55  self.batch_size = batch_size
    self.target_upd = target_update
    self.n_actions = n_actions
    self.step_count = 0

60  def select_action(self, state_np):
    if random.random() < self.epsilon:
        return random.randint(0, self.n_actions - 1)
    with torch.no_grad():
        q = self.q_net(torch.FloatTensor(state_np).unsqueeze(0))
65  return q.argmax().item()

    def greedy_action(self, state_np):

```

```

    with torch.no_grad():
        q = self.q_net(torch.FloatTensor(state_np).unsqueeze(0))
70     return q.argmax().item()

def push(self, s, a, r, ns, done):
    self.buffer.push(s, a, r, ns, float(done))

75 def learn(self):
    if len(self.buffer) < self.batch_size:
        return
    s, a, r, ns, d = self.buffer.sample(self.batch_size)

80     # Double DQN
    with torch.no_grad():
        best_a = self.q_net(ns).argmax(1, keepdim=True)
        q_target = r + self.gamma * (1 - d) * self.target_net(ns).gather(1,
            best_a).squeeze(1)

85     q_pred = self.q_net(s).gather(1, a.unsqueeze(1)).squeeze(1)
    loss = F.mse_loss(q_pred, q_target)
    self.opt.zero_grad()
    loss.backward()
    nn.utils.clip_grad_norm_(self.q_net.parameters(), 1.0)
90     self.opt.step()

    self.step_count += 1
    if self.step_count % self.target_upd == 0:
        self.target_net.load_state_dict(self.q_net.state_dict())

95     self.epsilon = max(self.eps_end, self.epsilon * self.eps_decay)

def train_dqn(n_episodes=DQN_EPISODES, print_every=200):
100     """Train DQN on the general (base) CryptoEnv."""
    env = CryptoEnv(train_features, train_prices, regime=train_regime)
    agent = DQNAgent()
    ep_rewards = []
    t0 = time.time()

105     for ep in range(1, n_episodes + 1):
        state = env.reset()
        done = False
        ep_r = 0.0
110         while not done:
            a = agent.select_action(state)
            ns, r, done, _ = env.step(a)
            agent.push(state, a, r, ns, done)
            agent.learn()
115             ep_r += r
            state = ns
        ep_rewards.append(ep_r)

        if ep % print_every == 0:
            avg = np.mean(ep_rewards[-print_every:])
            elapsed = time.time() - t0
            print(f" [DQN] Ep {ep:4d}/{n_episodes} "
                  f"AvgReward(last{print_every}): {avg:+.5f} "
120                  f"Epsilon: {agent.epsilon:.3f} Elapsed: {elapsed:.1f}s")

125     print(f" [DQN] Training complete. Total: {time.time()-t0:.1f}s")
    return agent, ep_rewards

```



```

130 print("Training DQN agent (Double DQN, 3-layer MLP, 800 episodes)...")
    dqn_agent, dqn_history = train_dqn(n_episodes=DQN_EPISODES)

    # Backtest DQN
    class DQNPolyWrapper:
135         """Wrap DQN to match PolicyNet interface expected by backtest()."""
        def __init__(self, dqn):
            self.dqn = dqn
        def greedy_action(self, state_np):
            return self.dqn.greedy_action(state_np)

140 dqn_res = backtest(DQNPolyWrapper(dqn_agent), test_features, test_prices,
                    regime=test_regime)
    backtest_results['DQN'] = dqn_res

    print(f"\n DQN          Return: {dqn_res['Total Return']:+.4f}  "
          f"Sharpe: {dqn_res['Sharpe Ratio']:+.3f}  "
          f"MaxDD: {dqn_res['Max Drawdown']:.4f}  "
          f"FinalPV: {dqn_res['Final Portfolio']:.4f}")

    print("\n" + "=" * 75)
150 print("COMPARISON TABLE: REINFORCE (3 trader types) vs DQN vs Buy&Hold")
    print("=" * 75)
    print(f"{'Agent':<22} {'Total Return':>14} {'Sharpe':>8} {'Max Drawdown':>14} {' "
          f"Final PV':>10}")
    print("-" * 75)
    for name, res in backtest_results.items():
155         print(f"{name:<22} {res['Total Return']:>14.4f} {res['Sharpe Ratio']:>8.3f} "
              f"{res['Max Drawdown']:>14.4f} {res['Final Portfolio']:>10.4f}")
    print("=" * 75)

```

Cell 9: SHAP Analysis — Kernel SHAP for all 3 trader types

```

def collect_states_greedy(policy_net, features, prices, n_steps=200):
    """
3   Run greedy policy on test env to collect states.
    Returns array of shape (n_steps, 12) using the raw (unnormalised) features
    plus portfolio fraction.
    We return two arrays:
        - states_norm: (n_steps, STATE_DIM) normalised features + frac
8       - states_raw12: (n_steps, 12) raw (unnormalised) feature values (for SHAP
        display)
    """
    T = len(prices)
    cash = 1.0
    crypto_held = 0.0
13   states_norm = []
    states_raw12 = []

    for t in range(min(n_steps, T - 1)):
        pv = cash + crypto_held * prices[t]
18       frac = (crypto_held * prices[t]) / (pv + 1e-9)
        feat_norm = features[t]
        state = np.concatenate([feat_norm, [frac]]).astype(np.float32)
        states_norm.append(state)
        states_raw12.append(feat_norm) # normalised features -- used as
        proxy

23   action = policy_net.greedy_action(state)
    if action == 2:
        cost = cash * (1 - TRANSACTION_COST)
        crypto_held += cost / prices[t]

```

```

28         cash          = 0.0
        elif action == 0:
            proceeds     = crypto_held * prices[t] * (1 - TRANSACTION_COST)
            cash         += proceeds
            crypto_held  = 0.0

33
        return np.array(states_norm), np.array(states_raw12)

def shap_predict_buy_prob(model, X):
    """
    SHAP-compatible prediction function.
    Takes (n, STATE_DIM) numpy array, returns (n,) Buy-probability.
    We pad the 12 or 13-dim input appropriately.
    """
43     n = X.shape[0]
    # X has 12 features (background only has feat columns, not frac)
    # We append a zero portfolio fraction to make STATE_DIM=13
    if X.shape[1] == 12:
        frac = np.zeros((n, 1), dtype=np.float32)
48         X_full = np.concatenate([X, frac], axis=1).astype(np.float32)
    else:
        X_full = X.astype(np.float32)

    with torch.no_grad():
53         t = torch.FloatTensor(X_full)
        probs = model(t).numpy() # (n, 3)
        return probs[:, 2]      # Buy probability (action=2)

58 print("Running SHAP KernelExplainer for each trader type...")
print("(Background: 30 states, Test: 100 states, nsamples=80)")
print()

shap_results = {} # {trader_type: {'shap_vals': array, 'mean_abs': array}}
63 N_BACKGROUND = 30
N_TEST         = 100
N_SHAP_SAMP    = 80

for ttype in ['arbitrageur', 'manipulator', 'herder']:
68     print(f"--- {ttype.upper()} ---")
    policy = agents[ttype].policy

    # Collect states
    states_norm, states_raw12 = collect_states_greedy(policy, test_features,
        test_prices, n_steps=200)
73     print(f" Collected {len(states_norm)} states from greedy rollout")

    bg_raw = states_raw12[:N_BACKGROUND] # (30, 12)
    tst_raw = states_raw12[N_BACKGROUND:N_BACKGROUND + N_TEST] # (100, 12)

78     def pred_fn(X):
        return shap_predict_buy_prob(policy, X)

    explainer = shap.KernelExplainer(pred_fn, bg_raw)
    shap_vals = explainer.shap_values(tst_raw, nsamples=N_SHAP_SAMP)
83     # shap_vals shape: (100, 12)

    mean_abs = np.abs(shap_vals).mean(axis=0) # (12,)

    # Ranked feature importance
88     ranked_idx = np.argsort(mean_abs)[::-1]
    print(f" Feature importance (ranked by mean |SHAP|):")

```

```

    for rank, idx in enumerate(ranked_idx):
        print(f"    #{rank+1:2d}  {FEATURE_COLS[idx]:>18s}  mean|SHAP|= {mean_abs
              [idx]:.6f}")

    shap_results[ttype] = {
        'shap_vals': shap_vals,
        'mean_abs': mean_abs,
        'ranked_idx': ranked_idx
    }
    print()

    print("SHAP analysis complete.")

# Summary: Top 3 features per trader type
print("\n" + "=" * 55)
print("TOP 3 FEATURES PER TRADER TYPE (mean |SHAP| for Buy prob)")
print("=" * 55)
for ttype in ['arbitrageur', 'manipulator', 'herder']:
    res = shap_results[ttype]
    print(f"\n{ttype.upper()}:")
    for rank in range(3):
        idx = res['ranked_idx'][rank]
        print(f"    #{rank+1}: {FEATURE_COLS[idx]:>18s}  {res['mean_abs'][idx]:.6f}")
    print("=" * 55)

```

Cell 10: Market Simulation — Crisis Frequency, Stablecoin Stability, Market Efficiency

```

import yfinance as yf

# Download USDT-USD for stablecoin stability (actual peg deviation data)
_start = ftest.index[0].strftime('%Y-%m-%d')
_end = ftest.index[-1].strftime('%Y-%m-%d')
usdt_raw = yf.download('USDT-USD', start=_start, end=_end, interval='1h',
                      auto_adjust=True, progress=False)

usdt_close = usdt_raw['Close']
if isinstance(usdt_close, pd.DataFrame): usdt_close = usdt_close.iloc[:,0]
if usdt_close.index.tzinfo: usdt_close.index = usdt_close.index.tz_convert(None)
usdt_peg_dev = (usdt_close - 1.0).abs() * 100 # % deviation from $1.00 peg
usdt_arr = usdt_peg_dev.reindex(ftest.index, method='nearest',
                                tolerance=pd.Timedelta('2h')).fillna(
                                    usdt_peg_dev.mean()).values

print(f"USDT aligned: {(~np.isnan(usdt_arr)).sum()} hours, mean peg dev: {
      usdt_arr.mean():.4f}%")

feat_test = ftest.values.astype(np.float32)
price_test = ptest.values.astype(np.float32)
regime_arr = rtest.values
SIM_STEPS = min(500, len(feat_test) - 2)

def a2sig(a): return {1: 1., 0: 0., 2: -1.}[a]

def simulate_market(w_arb, w_man, w_hrd):
    """
    Simulate 500 hours on test data under a given trader-type composition.
    Returns: crisis_frequency, stablecoin_stability (USDT-based),
            market_efficiency.
    """
    agents = [(arb_agent, w_arb), (man_agent, w_man), (hrd_agent, w_hrd)]
    crypto_frac = 0.0

```

```

crisis_steps = 0
sim_rets, signals, stab_vals = [], [], []
33 price = price_test[0]

for t in range(SIM_STEPS):
    state = np.append(feas_test[t], crypto_frac).astype(np.float32)
    agg_sig = sum(w * a2sig(ag.act(state, greedy=True)) for ag, w in agents)
38 signals.append(agg_sig)

    # Price with micro market-impact (0.05% per unit signal)
    impact = agg_sig * 0.0005
    new_price = price * (1 + feas_test[t, FEAT_COLS.index('ret_1h')] + impact)
43 ret = (new_price - price) / (price + 1e-9)
    sim_rets.append(ret)
    price = new_price

    # Update portfolio exposure based on aggregate action
48 if agg_sig > 0.3: crypto_frac = min(crypto_frac + 0.1, 1.0)
    elif agg_sig < -0.3: crypto_frac = max(crypto_frac - 0.1, 0.0)

    # Crisis: confirmed decline regime + strong collective sell signal
    if regime_arr[t] == 'decline' and agg_sig < -0.3:
53 crisis_steps += 1

    stab_vals.append(usdt_arr[t])

sigs = np.array(signals)
58 # Stablecoin stability: USDT peg deviation DURING high-stress periods
stressed = np.abs(sigs) > 0.3
usdt_stress = np.array(stab_vals)[stressed].mean() if stressed.sum() > 5 else
    np.array(stab_vals).mean()
stablecoin_stability = 1.0 / (usdt_stress + 1e-6) # higher = more stable

63 sim_rets = np.array(sim_rets)
lag1_ac = np.corrcoef(sim_rets[:-1], sim_rets[1:])[0, 1] if len(sim_rets) >
    2 else 0.
market_efficiency = 1.0 - abs(lag1_ac)

return {
68     'crisis_freq': crisis_steps / SIM_STEPS,
    'stablecoin_stability': stablecoin_stability,
    'market_efficiency': market_efficiency
}

73 compositions = [
    (1.0, 0.0, 0.0), (0.0, 1.0, 0.0), (0.0, 0.0, 1.0),
    (0.33, 0.33, 0.34), (0.6, 0.2, 0.2), (0.2, 0.6, 0.2), (0.2, 0.2, 0.6)
]
comp_labels = ["All Arb", "All Manip", "All Herd", "Equal", "Arb-Dom", "Manip-Dom", "Herd-Dom"]
78 sim_results = {}
for (wa, wm, wh), lb in zip(compositions, comp_labels):
    sim_results[lb] = simulate_market(wa, wm, wh)

83 print("\nMarket Simulation Results:")
print(f"{'Composition':12s} {'Crisis Freq':>12s} {'USDT Stability':>15s} {'Mkt Efficiency':>14s}")
for lb in comp_labels:
    r = sim_results[lb]
    print(f"{lb:12s} {r['crisis_freq']:>12.4f} {r['stablecoin_stability']:>15.4f} {r['market_efficiency']:>14.4f}")

```

Cell 11: Three-panel market metrics figure

```

bar_colors = {
    "All Arb":      '#2166AC',    # blue
    "All Manip":    '#D6604D',    # red
    "All Herd":      '#F4A582',    # orange
    "Equal":         '#762A83',    # purple
    "Arb-Dom":       '#4393C3',    # light blue
    "Manip-Dom":     '#B2182B',    # dark red
    "Herd-Dom":      '#E08214',    # dark orange
}

crisis_vals = [sim_results[l]['crisis_freq'] for l in comp_labels]
stab_vals   = [sim_results[l]['price_stability'] for l in comp_labels]
eff_vals    = [sim_results[l]['market_efficiency'] for l in comp_labels]
colors      = [bar_colors[l] for l in comp_labels]
x           = np.arange(len(comp_labels))

equal_idx = comp_labels.index("Equal")

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle("Market Composition vs. Stability Metrics\n(REINFORCE Multi-Trader\nSimulation)",
             fontsize=14, fontweight='bold', y=1.02)

# Panel 1: Crisis Frequency
ax = axes[0]
bars = ax.bar(x, crisis_vals, color=colors, edgecolor='black', linewidth=0.7,
             alpha=0.85)
ax.axhline(y=crisis_vals[equal_idx], color='purple', linestyle='--', linewidth=
             =1.8,
            label=f'Equal mix baseline ({crisis_vals[equal_idx]:.3f})')
ax.set_xticks(x); ax.set_xticklabels(comp_labels, rotation=30, ha='right',
            fontsize=9)
ax.set_ylabel("Crisis Frequency (fraction of steps)")
ax.set_title("Crisis Frequency\n(lower is better)", fontsize=11)
ax.legend(fontsize=8)
ax.set_ylim(0, max(crisis_vals) * 1.35 if max(crisis_vals) > 0 else 0.1)
for bar, v in zip(bars, crisis_vals):
    ax.text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.001,
            f'{v:.3f}', ha='center', va='bottom', fontsize=8)

# Panel 2: Price Stability
ax = axes[1]
bars = ax.bar(x, stab_vals, color=colors, edgecolor='black', linewidth=0.7, alpha
             =0.85)
ax.axhline(y=stab_vals[equal_idx], color='purple', linestyle='--', linewidth=1.8,
            label=f'Equal mix baseline ({stab_vals[equal_idx]:.2f})')
ax.set_xticks(x); ax.set_xticklabels(comp_labels, rotation=30, ha='right',
            fontsize=9)
ax.set_ylabel("Price Stability (1 / std of agg. signals)")
ax.set_title("Price Stability\n(higher is better)", fontsize=11)
ax.legend(fontsize=8)
ax.set_ylim(0, max(stab_vals) * 1.2)
for bar, v in zip(bars, stab_vals):
    ax.text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.02,
            f'{v:.2f}', ha='center', va='bottom', fontsize=8)

# Panel 3: Market Efficiency
ax = axes[2]
bars = ax.bar(x, eff_vals, color=colors, edgecolor='black', linewidth=0.7, alpha
             =0.85)
ax.axhline(y=eff_vals[equal_idx], color='purple', linestyle='--', linewidth=1.8,

```

```

        label=f'Equal mix baseline ({eff_vals[equal_idx]:.3f})')
ax.set_xticks(x); ax.set_xticklabels(comp_labels, rotation=30, ha='right',
    fontsize=9)
ax.set_ylabel("Market Efficiency (1 - |lag-1 autocorr|)")
58 ax.set_title("Market Efficiency\n(higher is better)", fontsize=11)
ax.legend(fontsize=8)
ax.set_ylim(0, 1.15)
for bar, v in zip(bars, eff_vals):
    ax.text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.005,
63         f'{v:.3f}', ha='center', va='bottom', fontsize=8)

# Shared legend for trader types
legend_patches = [
    mpatches.Patch(color='#2166AC', label='Arbitrageur-heavy'),
68    mpatches.Patch(color='#D6604D', label='Manipulator-heavy'),
    mpatches.Patch(color='#F4A582', label='Herder-heavy'),
    mpatches.Patch(color='#762A83', label='Mixed'),
]
fig.legend(handles=legend_patches, loc='lower center', ncol=4,
73         bbox_to_anchor=(0.5, -0.08), fontsize=9, frameon=True)

plt.tight_layout()
out_path = os.path.join(OUT_DIR, 'part3_market_metrics.png')
plt.savefig(out_path, dpi=150, bbox_inches='tight')
78 plt.show()
print(f"Saved: {out_path}")

```

Cell 12: SHAP Summary Bar Charts (3 subplots, one per trader type)

```

1 fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle("SHAP Feature Importance for Buy-Action Probability\n(Mean |SHAP|
    across 100 test states)",
    fontsize=13, fontweight='bold')

trader_colors = {
6     'arbitrageur': '#2166AC',
     'manipulator': '#D6604D',
     'herder':      '#E08214',
}

11 for ax, ttype in zip(axes, ['arbitrageur', 'manipulator', 'herder']):
    res = shap_results[ttype]
    mean_abs = res['mean_abs'] # (12,)
    ranked = res['ranked_idx']

16     # Sort features by mean |SHAP| descending
    sorted_vals = mean_abs[ranked]
    sorted_names = [FEATURE_COLS[i] for i in ranked]

    ypos = np.arange(len(FEATURE_COLS))
21     ax.barh(ypos, sorted_vals, color=trader_colors[ttype], alpha=0.85,
        edgecolor='black', linewidth=0.5)
    ax.set_yticks(ypos)
    ax.set_yticklabels(sorted_names, fontsize=9)
    ax.invert_yaxis() # top = most important
26     ax.set_xlabel("Mean |SHAP| value")
    ax.set_title(f"{ttype.capitalize()}\n(top: {sorted_names[0]})", fontsize=11)

    # Annotate top 3
    for i in range(min(3, len(sorted_vals))):
31         ax.text(sorted_vals[i] + sorted_vals[0] * 0.01,
            ypos[i], f'{sorted_vals[i]:.5f}',

```

```

        va='center', fontsize=7.5, color='black')

plt.tight_layout()
36 shap_out = os.path.join(OUT_DIR, 'part3_shap.png')
plt.savefig(shap_out, dpi=150, bbox_inches='tight')
plt.show()
print(f"Saved: {shap_out}")

```

Cell 13: Training Curves — 100-episode rolling average for all 3 REINFORCE types

```

1 fig, ax = plt.subplots(figsize=(12, 5))

color_map = {'arbitrageur': '#2166AC', 'manipulator': '#D6604D', 'herder': '#E08214'}
roll_window = 100

6 for ttype, history in reward_histories.items():
    x_eps = np.arange(1, len(history) + 1)
    raw = np.array(history)
    # Rolling average using pandas for clean handling of edges
    rolled = pd.Series(raw).rolling(window=roll_window, min_periods=1).mean().values

11     ax.plot(x_eps, raw, alpha=0.15, color=color_map[ttype], linewidth=0.6)
    ax.plot(x_eps, rolled, color=color_map[ttype], linewidth=2.0,
            label=f'{ttype.capitalize()} ({roll_window}-ep MA)')

16 ax.set_xlabel("Episode", fontsize=11)
ax.set_ylabel("Cumulative Episode Reward", fontsize=11)
ax.set_title("REINFORCE Training Curves by Trader Type\n"
            "(light: raw, solid: 100-episode rolling average)", fontsize=12)
ax.legend(fontsize=10)

21 ax.grid(True, alpha=0.3)
ax.axhline(y=0, color='black', linewidth=0.8, linestyle='--', alpha=0.5)

plt.tight_layout()
training_out = os.path.join(OUT_DIR, 'part3_training.png')
26 plt.savefig(training_out, dpi=150, bbox_inches='tight')
plt.show()
print(f"Saved: {training_out}")

```

Cell 14: Summary Results Table (formatted DataFrame)

```

metrics_data = []
2 for name, res in backtest_results.items():
    metrics_data.append({
        'Agent': name,
        'Total Return': f"{res['Total Return']:+.4f}",
        'Sharpe Ratio': f"{res['Sharpe Ratio']:+.3f}",
7         'Max Drawdown': f"{res['Max Drawdown']:.4f}",
        'Final Portfolio': f"{res['Final Portfolio']:.4f}",
    })

summary_df = pd.DataFrame(metrics_data).set_index('Agent')

12 print("=" * 68)
print("SUMMARY: All Agents -- Backtest Performance on Test Set")
print("=" * 68)
print(summary_df.to_string())
17 print("=" * 68)

```

```

print()

# Also show raw numeric summary
print("\nDetailed numeric summary:")
22 metrics_num = []
for name, res in backtest_results.items():
    metrics_num.append({
        'Agent': name,
        'Total Return': res['Total Return'],
27 'Sharpe Ratio': res['Sharpe Ratio'],
        'Max Drawdown': res['Max Drawdown'],
        'Final Portfolio': res['Final Portfolio'],
    })
df_num = pd.DataFrame(metrics_num).set_index('Agent')
32 print(df_num.round(4).to_string())

# Best agent by Sharpe
best_sharpe = df_num['Sharpe Ratio'].idxmax()
best_return = df_num['Total Return'].idxmax()
37 print(f"\nBest Sharpe: {best_sharpe} ({df_num.loc[best_sharpe, 'Sharpe
    Ratio']:.3f})")
print(f"Best Total Return: {best_return} ({df_num.loc[best_return, 'Total Return
    ']:.4f})")

```